

5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.

a) Label the first 50 points {x1,...,x50} as follows: if ($x_i \leq 0.5$), then $x_i \in \text{Class1}$, else $x_i \in \text{Class1}$

b) Classify the remaining points, x51,...,x100 using KNN. Perform this for $k=1,2,3,4,5,20,30$

1. Objectives

- Generate 100 random values in the range [0, 1].
- Label the first 50 data points using a threshold rule:
 - $x \leq 0.5 \rightarrow \text{Class1}$
 - $x > 0.5 \rightarrow \text{Class2}$
- Use the k-Nearest Neighbour (KNN) algorithm to classify the remaining 50 points.
- Analyze classification performance for multiple values of:
 $k = 1,2,3,4,5,20,30$

Visualize classification results using scatter plots.

2. Step-by-Step Analysis of the Program

Step 1: Import Libraries

- numpy $\rightarrow$ for generating random data.
- matplotlib.pyplot $\rightarrow$ for visualization.
- Counter $\rightarrow$ for majority voting in KNN.

Step 2: Data Generation

- np.random.rand(100) generates 100 random values uniformly in [0, 1].

Step 3: Label Training Data

- First 50 points are labeled:

labels = ["Class1" if $x \leq 0.5$ else "Class2"]

- This creates a clear decision boundary at 0.5.

Step 4: Distance Function

- Euclidean distance (1D case):

$$d = |x_1 - x_2|$$

- Implemented using:

return abs(x1 - x2)

Step 5: KNN Classifier Logic

For each test point:

1. Compute distance to all training points.
2. Sort distances in ascending order.
3. Select k nearest neighbors.
4. Extract their labels.
5. Use majority voting:

Counter(labels).most_common(1)

Step 6: Train-Test Split

- Training data $\rightarrow$ first 50 points.
- Testing data $\rightarrow$ last 50 points.

Step 7: Classification for Different k Values

- Loop through:

k_values = [1,2,3,4,5,20,30]

- Store predictions in a dictionary.

Step 8: Output Display

- Prints classification for each test point:

Point x51 (value: 0.7321) is classified as Class2

Step 9: Visualization

- Training data plotted at $y = 0$

- Test data plotted at $y = 1$
- Colors:
 - Blue $\rightarrow$ Class1
 - Red $\rightarrow$ Class2
- Separate graph generated for each k value.

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
data = np.random.rand(100)
labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]
def euclidean_distance(x1, x2):
    return abs(x1 - x2)
def knn_classifier(train_data, train_labels, test_point, k):
    distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i]) for i in
range(len(train_data))]

    distances.sort(key=lambda x: x[0])
    k_nearest_neighbors = distances[:k]
    k_nearest_labels = [label for _, label in k_nearest_neighbors]
    return Counter(k_nearest_labels).most_common(1)[0][0]
train_data = data[:50]
train_labels = labels
test_data = data[50:]
k_values = [1, 2, 3, 4, 5, 20, 30]
print("--- k-Nearest Neighbors Classification ---")
print("Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 ->
Class2)")
print("Testing dataset: Remaining 50 points to be classified\n")
results = {}

for k in k_values:
    print(f"Results for k = {k}:")
    classified_labels = [knn_classifier(train_data, train_labels, test_point, k) for test_point in test_data]
    results[k] = classified_labels
    for i, label in enumerate(classified_labels, start=51):
        print(f"Point x{i} (value: {test_data[i - 51]:.4f}) is classified as {label}")
    print("\n")
print("Classification complete.\n")

for k in k_values:
    classified_labels = results[k]
    class1_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class1"]
    class2_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class2"]
    plt.figure(figsize=(10, 6))
    plt.scatter(train_data, [0] * len(train_data), c=["blue" if label == "Class1" else "red" for label in
train_labels],
                label="Training Data", marker="o")
    plt.scatter(class1_points, [1] * len(class1_points), c="blue", label="Class1 (Test)", marker="x")
    plt.scatter(class2_points, [1] * len(class2_points), c="red", label="Class2 (Test)", marker="x")
    plt.title(f"k-NN Classification Results for k = {k}")
    plt.xlabel("Data Points")
    plt.ylabel("Classification Level")
    plt.legend()
    plt.grid(True)
    plt.show()

```

3. Key Observations

Effect of k Value

- $k = 1$
 - Very sensitive to nearest point.
 - High variance (overfitting).
- $k = 2-5$
 - More stable predictions.
 - Better balance between bias and variance.
- $k = 20, 30$
 - Predictions become smoother.
 - Majority class dominates (underfitting).

Dataset Nature

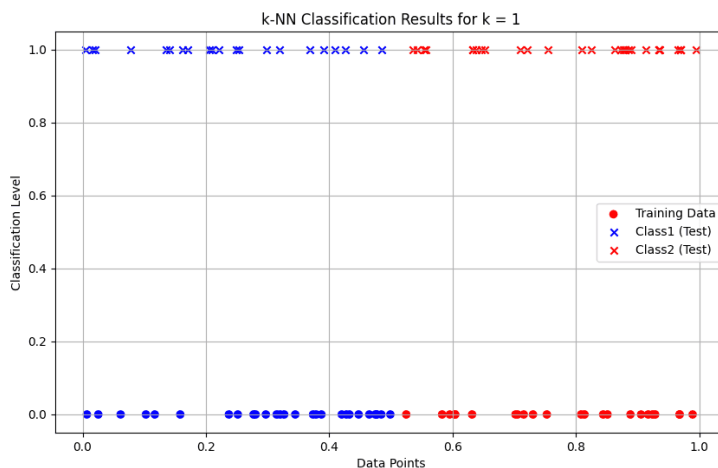
- Since labeling is deterministic, KNN should approximate:
 $x = 0.5$ boundary
- Errors mainly occur near the boundary.

4. Advantages of the Program

- Simple and intuitive implementation
- No training phase (lazy learning)
- Works well for small datasets
- Easily adaptable for higher dimensions
- Visualization improves interpretability

5. Limitations

- Computationally expensive
 - Distance calculated for every test point
- Sensitive to k selection
- Affected by noise (especially for $k=1$)
- Memory intensive
 - Stores entire dataset
- Not efficient for large-scale datasets



--- k-Nearest Neighbors Classification ---

Training dataset: First 50 points labeled based on the rule ($x \leq 0.5 \rightarrow \text{Class1}$, $x > 0.5 \rightarrow \text{Class2}$)

Testing dataset: Remaining 50 points to be classified

Results for $k = 1$:

Point x51 (value: 0.2059) is classified as Class1

Point x52 (value: 0.2535) is classified as Class1

Point x53 (value: 0.4856) is classified as Class1

Point x54 (value: 0.9651) is classified as Class2

Point x55 (value: 0.3906) is classified as Class1

Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class2
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class2
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class2
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for $k = 2$:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2

Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class2
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class2
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class2
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for $k = 3$:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class2
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class2

Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class2
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for $k = 4$:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class2
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class2
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1

Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class2
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for $k = 5$:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class2
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class1
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class2

Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for k = 20:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class1
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class1
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class1
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1

Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1
Point x92 (value: 0.3693) is classified as Class1
Point x93 (value: 0.4087) is classified as Class1
Point x94 (value: 0.8103) is classified as Class2
Point x95 (value: 0.0773) is classified as Class1
Point x96 (value: 0.8792) is classified as Class2
Point x97 (value: 0.9138) is classified as Class2
Point x98 (value: 0.5567) is classified as Class2
Point x99 (value: 0.8625) is classified as Class2
Point x100 (value: 0.9363) is classified as Class2

Results for $k = 30$:

Point x51 (value: 0.2059) is classified as Class1
Point x52 (value: 0.2535) is classified as Class1
Point x53 (value: 0.4856) is classified as Class1
Point x54 (value: 0.9651) is classified as Class2
Point x55 (value: 0.3906) is classified as Class1
Point x56 (value: 0.8903) is classified as Class2
Point x57 (value: 0.9695) is classified as Class2
Point x58 (value: 0.2206) is classified as Class1
Point x59 (value: 0.0203) is classified as Class1
Point x60 (value: 0.1619) is classified as Class1
Point x61 (value: 0.6461) is classified as Class2
Point x62 (value: 0.6523) is classified as Class2
Point x63 (value: 0.8728) is classified as Class2
Point x64 (value: 0.5435) is classified as Class1
Point x65 (value: 0.8246) is classified as Class2
Point x66 (value: 0.9347) is classified as Class2
Point x67 (value: 0.5361) is classified as Class1
Point x68 (value: 0.7215) is classified as Class2
Point x69 (value: 0.9703) is classified as Class2
Point x70 (value: 0.8764) is classified as Class2
Point x71 (value: 0.7543) is classified as Class2
Point x72 (value: 0.1406) is classified as Class1
Point x73 (value: 0.1349) is classified as Class1
Point x74 (value: 0.9705) is classified as Class2
Point x75 (value: 0.2985) is classified as Class1
Point x76 (value: 0.9948) is classified as Class2
Point x77 (value: 0.4551) is classified as Class1
Point x78 (value: 0.2101) is classified as Class1
Point x79 (value: 0.5542) is classified as Class1
Point x80 (value: 0.3202) is classified as Class1
Point x81 (value: 0.6325) is classified as Class2
Point x82 (value: 0.9345) is classified as Class2
Point x83 (value: 0.0156) is classified as Class1
Point x84 (value: 0.8859) is classified as Class2
Point x85 (value: 0.2495) is classified as Class1
Point x86 (value: 0.6380) is classified as Class2
Point x87 (value: 0.7095) is classified as Class2
Point x88 (value: 0.4259) is classified as Class1
Point x89 (value: 0.0052) is classified as Class1
Point x90 (value: 0.6322) is classified as Class2
Point x91 (value: 0.1701) is classified as Class1

Point x92 (value: 0.3693) is classified as Class1
 Point x93 (value: 0.4087) is classified as Class1
 Point x94 (value: 0.8103) is classified as Class2
 Point x95 (value: 0.0773) is classified as Class1
 Point x96 (value: 0.8792) is classified as Class2
 Point x97 (value: 0.9138) is classified as Class2
 Point x98 (value: 0.5567) is classified as Class1
 Point x99 (value: 0.8625) is classified as Class2
 Point x100 (value: 0.9363) is classified as Class2
 Classification complete.

6. Conclusion

- The program successfully demonstrates KNN classification in a 1D space.
- It highlights how:
 - Small $k \rightarrow$ overfitting
 - Large $k \rightarrow$ underfitting
- Optimal performance is typically achieved for moderate values of k (3–5).
- The visualization clearly shows how classification changes with different k values.
- KNN is effective for simple problems but requires careful parameter tuning and is less scalable for large datasets.

6. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

1. Objectives

- Implement a non-parametric regression algorithm (Locally Weighted Regression).
- Fit a smooth curve to a non-linear dataset.
- Use a Gaussian kernel to assign weights based on proximity.
- Analyze the effect of the bandwidth parameter (τ) on model behavior.
- Visualize the fitted curve against noisy training data.

2. Step-by-Step Analysis of the Program

Step 1: Import Libraries

- numpy $\rightarrow$ numerical computations.
- matplotlib.pyplot $\rightarrow$ plotting graphs.

Step 2: Define Gaussian Kernel

- Weight function:

$$w = \exp \left(-\frac{(x-x_i)^2}{2\tau^2} \right)$$

- Purpose:
 - Assigns higher weight to nearby points
 - Distant points get negligible influence

Step 3: Locally Weighted Regression Function

For each query point x :

1. Compute weights for all training points:
weights = gaussian_kernel(x, Xi, tau)
2. Construct diagonal weight matrix:
 $W = \text{diag}(\text{weights})$
3. Compute parameters using weighted least squares:

$$\theta = (X^T W X)^{-1} X^T W y$$

4. Predict output:

$$y = x \cdot \theta$$

Step 4: Generate Dataset

- Input:

$$X = [0, 2\pi]$$

- Output:

$$y = \sin(X) + \text{noise}$$

- Noise adds realism (non-perfect data).

Step 5: Add Bias Term

- Convert:

$$X \rightarrow [1, X]$$

- Enables linear model to fit locally.

Step 6: Prediction Points

- Generate dense test points (x_{test}) for smooth curve plotting.

Step 7: Apply LWR

- For each test point:
 - Compute a separate local model
- This makes LWR:
 - Lazy learning
 - No global model is stored

Step 8: Plotting

- Red dots → noisy training data
- Blue curve → LWR fitted curve

```
import numpy as np
import matplotlib.pyplot as plt

def gaussian_kernel(x, xi, tau):
    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))

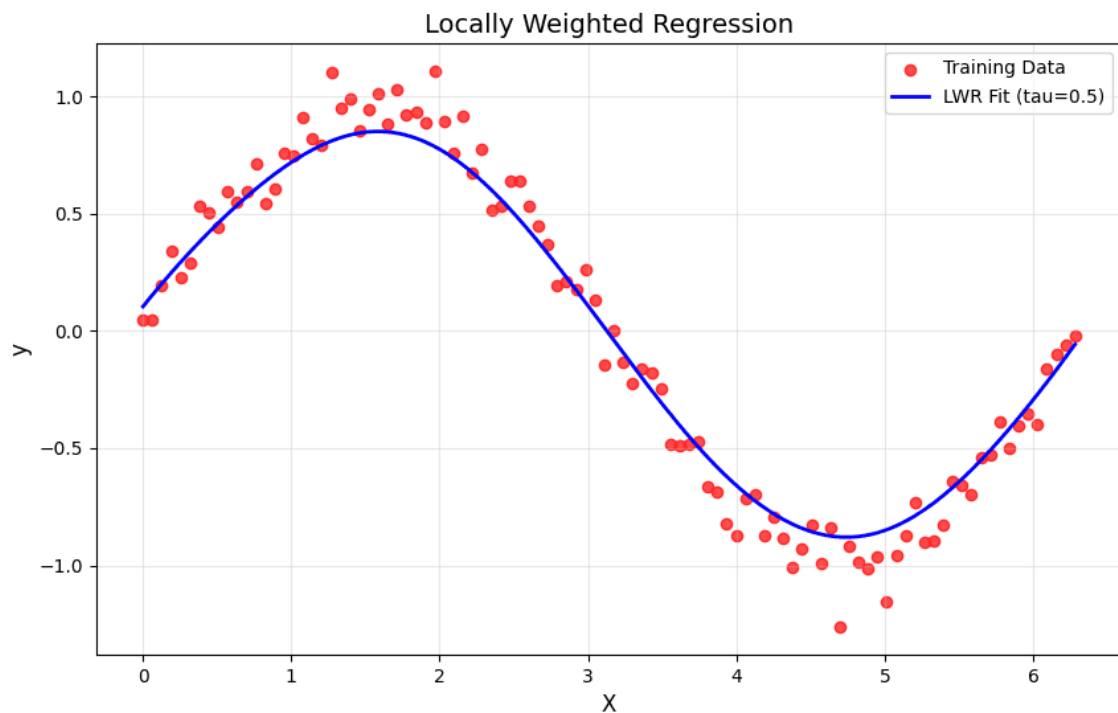
def locally_weighted_regression(x, X, y, tau):
    m = X.shape[0]
    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])
    W = np.diag(weights)
    X_transpose_W = X.T @ W
    theta = np.linalg.inv(X_transpose_W @ X) @ X_transpose_W @ y
    return x @ theta

np.random.seed(42)
X = np.linspace(0, 2 * np.pi, 100)
y = np.sin(X) + 0.1 * np.random.randn(100)
X_bias = np.c_[np.ones(X.shape), X]

x_test = np.linspace(0, 2 * np.pi, 200)
x_test_bias = np.c_[np.ones(x_test.shape), x_test]
tau = 0.5
y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])

plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)
plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)
```

```
plt.xlabel('X', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('Locally Weighted Regression', fontsize=14)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()
```



3. Key Observations

Behavior of LWR Fit

- ☐ The model **closely follows the sine curve** despite noise.
- ☐ Captures **local patterns effectively**.

Effect of Bandwidth (τ)

τ Value	Behavior
Small τ	Very flexible, may overfit noise
Medium τ (0.5 used)	Good balance
Large τ	Over-smooth, behaves like linear regression

Key Insights

- Each prediction is influenced mostly by **nearby points**.
- No single global equation represents the data.
- Works well for **non-linear relationships**.

4. Advantages

- **Non-parametric** (no assumption about data distribution)
- Captures **complex, non-linear patterns**
- Highly **flexible and adaptive**
- Works well with **small datasets**
- Smooth interpolation between points

5. Limitations

- **Computationally expensive**
 - Model trained separately for each query point

- Requires storing **entire dataset**
- Sensitive to:
 - Bandwidth parameter (τ)
 - Noise (for very small τ)
- Matrix inversion may be:
 - **Numerically unstable** if data is ill-conditioned
- Not suitable for **large-scale datasets**

6. Conclusion

- The program successfully demonstrates **Locally Weighted Regression** as a powerful **non-parametric technique**.
- It effectively models **non-linear relationships** like the sine function.
- The choice of **bandwidth (τ)** is critical:
 - Small $\tau \rightarrow$ overfitting
 - Large $\tau \rightarrow$ underfitting
- LWR is ideal for:
 - Small datasets
 - Situations requiring **high flexibility**
- However, due to its computational cost, it is **less practical for large datasets**.

7. Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

1. Objectives

- Implement and compare:
 - Linear Regression (simple linear relationship)
 - Polynomial Regression (non-linear relationship)
- Use real-world datasets:
 - California Housing Dataset for linear regression
 - Auto MPG Dataset for polynomial regression
- Evaluate model performance using:
 - Mean Squared Error (MSE)
 - R^2 Score
- Visualize actual vs predicted values.

2. Step-by-Step Analysis

A. Linear Regression (California Housing Dataset)

Step 1: Load Dataset

- Uses `fetch_california_housing()`
- Feature selected:
 - AveRooms (average number of rooms)
- Target:
 - Median house value

Step 2: Train-Test Split

- 80% training, 20% testing
- Ensures unbiased evaluation

Step 3: Model Training

- Apply:

`LinearRegression()`

- Fits a straight-line model:

$$y = mx + c$$

Step 4: Prediction

- Predict house prices on test data.

Step 5: Visualization

- Scatter plot $\rightarrow$ actual values

- Line plot → predicted values

Step 6: Evaluation Metrics

- MSE → error magnitude
- R^2 Score → goodness of fit

B. Polynomial Regression (Auto MPG Dataset)

Step 1: Load Dataset

- Reads data from UCI repository
- Feature:
 - displacement
- Target:
 - mpg (fuel efficiency)

Step 2: Data Cleaning

- Removes missing values (dropna())

Step 3: Train-Test Split

- Same 80-20 split

Step 4: Model Pipeline

- Pipeline includes:
 1. PolynomialFeatures(degree=2) → creates quadratic terms
 2. StandardScaler() → normalizes features
 3. LinearRegression() → fits transformed data
- Model equation becomes:

$$y = ax^2 + bx + c$$

Step 5: Prediction

- Predict mpg values for test data.

Step 6: Visualization

- Scatter plots for:
 - Actual values
 - Predicted values

Step 7: Evaluation

- Compute MSE and R^2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

def linear_regression_california():
    housing = fetch_california_housing(as_frame=True)
    X = housing.data[["AveRooms"]]
    y = housing.target

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    plt.scatter(X_test, y_test, color="blue", label="Actual")
```

```

plt.plot(X_test, y_pred, color="red", label="Predicted")
plt.xlabel("Average number of rooms (AveRooms)")
plt.ylabel("Median value of homes ($100,000)")
plt.title("Linear Regression - California Housing Dataset")
plt.legend()
plt.show()

print("Linear Regression - California Housing Dataset")
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R^2 Score:", r2_score(y_test, y_pred))

def polynomial_regression_auto_mpg():
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
    column_names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration",
"model_year", "origin"]
    data = pd.read_csv(url, sep='\s+', names=column_names, na_values="?")
    data = data.dropna()

    X = data["displacement"].values.reshape(-1, 1)
    y = data["mpg"].values

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    poly_model = make_pipeline(PolynomialFeatures(degree=2), StandardScaler(),
LinearRegression())
    poly_model.fit(X_train, y_train)

    y_pred = poly_model.predict(X_test)

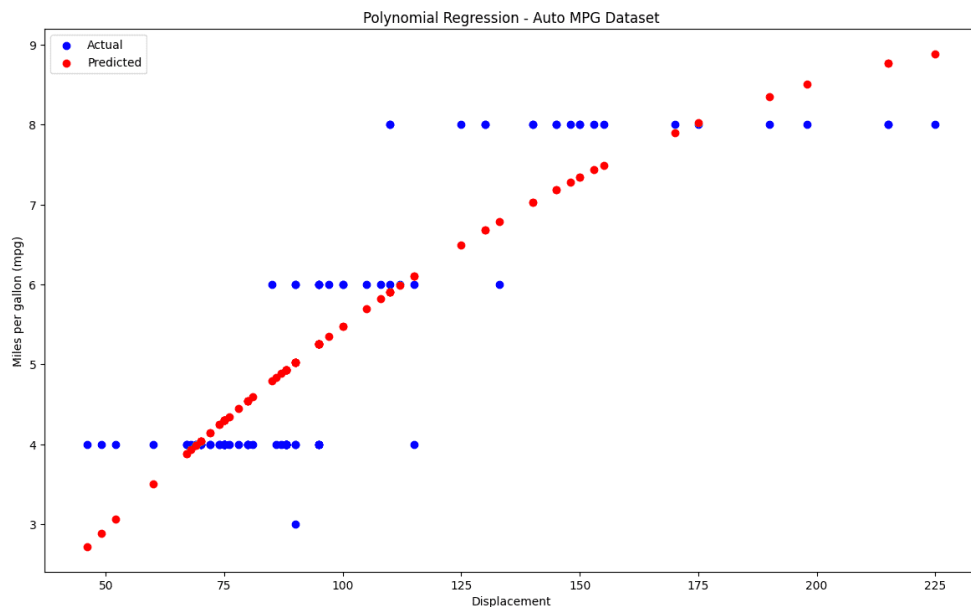
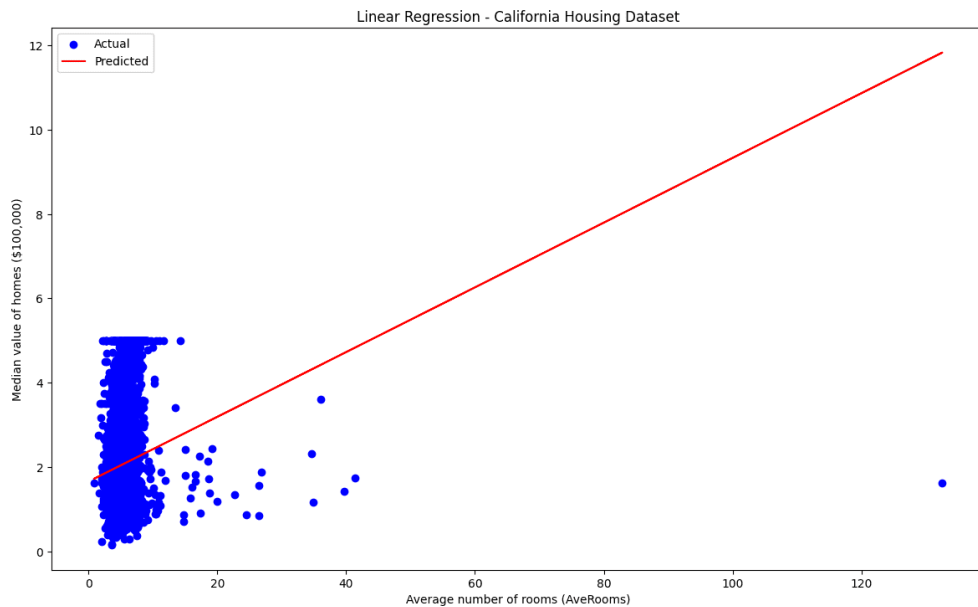
    plt.scatter(X_test, y_test, color="blue", label="Actual")
    plt.scatter(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Displacement")
    plt.ylabel("Miles per gallon (mpg)")
    plt.title("Polynomial Regression - Auto MPG Dataset")
    plt.legend()
    plt.show()

    print("Polynomial Regression - Auto MPG Dataset")
    print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

if __name__ == "__main__":
    print("Demonstrating Linear Regression and Polynomial Regression\n")
    linear_regression_california()
    polynomial_regression_auto_mpg()

```

OUTPUT:



Linear Regression - California Housing Dataset

Mean Squared Error: 1.2923314440807299

R^2 Score: 0.013795337532284901

Polynomial Regression - Auto MPG Dataset

Mean Squared Error: 0.743149055720586

Demonstrating Linear Regression and Polynomial Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree
```

```
data = load_breast_cancer()
```



```

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Model Accuracy: {accuracy * 100:.2f}%')
new_sample = np.array([X_test[0]])
prediction = clf.predict(new_sample)

prediction_class = "Benign" if prediction == 1 else "Malignant"
print(f'Predicted Class for the new sample: {prediction_class}')

plt.figure(figsize=(12,8))
tree.plot_tree(clf, filled=True, feature_names=data.feature_names, class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
R^2 Score: 0.7505650609469626

```

3. Key Observations

Linear Regression

- Captures linear trends only
- Works reasonably if relationship is approximately linear
- May underfit complex data

Polynomial Regression

- Captures non-linear relationships
- Provides better fit for curved data patterns
- Risk of overfitting if degree is high

Model Comparison

Aspect	Linear Regression	Polynomial Regression
Relationship	Linear	Non-linear
Complexity	Simple	Moderate
Flexibility	Low	High
Overfitting Risk	Low	High

4. Advantages

Linear Regression

- Simple and fast
- Easy to interpret
- Works well for linear data

Polynomial Regression

- Captures complex patterns
- More flexible than linear regression
- Better accuracy for non-linear datasets

5. Limitations

Linear Regression

- Cannot model non-linear relationships
- Sensitive to outliers
- May underfit real-world data

Polynomial Regression

- Overfitting risk (especially high degree)
- Computationally more expensive
- Harder to interpret
- Requires feature scaling

6. Conclusion

- The program effectively demonstrates the difference between:
 - **Linear models** (simple, interpretable)
 - **Polynomial models** (flexible, powerful)
- Results show:
 - Linear regression works well for **simple trends**
 - Polynomial regression is better for **curved relationships**
- Model selection depends on:
 - Nature of data
 - Trade-off between bias and variance

8. Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

1. Objectives

- Implement a **Decision Tree Classifier** for supervised learning.
- Use the Breast Cancer Dataset to classify tumors.
- Train the model and evaluate its **accuracy**.
- Predict the class (Benign/Malignant) for a **new sample**.
- Visualize the **decision tree structure**.

2. Step-by-Step Analysis

Step 1: Import Libraries

- numpy → numerical operations
- matplotlib → visualization
- sklearn modules → dataset, model, evaluation

Step 2: Load Dataset

- load_breast_cancer() provides:
 - Features (e.g., radius, texture, smoothness)
 - Target:
 - 0 → Malignant
 - 1 → Benign

Step 3: Define Features and Labels

X = data.data

y = data.target

Step 4: Train-Test Split

- 80% training, 20% testing:
train_test_split(..., test_size=0.2)
- Ensures unbiased evaluation.

Step 5: Model Creation

DecisionTreeClassifier(random_state=42)

- Builds a tree using:
 - Feature splits
 - Threshold values
- Objective: maximize **information gain** (or reduce impurity)

Step 6: Model Training

```
clf.fit(X_train, y_train)
```

- Learns decision rules from training data.

Step 7: Prediction

```
y_pred = clf.predict(X_test)
```

- Predicts classes for unseen data.

Step 8: Accuracy Evaluation

```
accuracy_score(y_test, y_pred)
```

- Measures classification performance.

Step 9: Classifying a New Sample

```
new_sample = np.array([X_test[0]])
```

```
prediction = clf.predict(new_sample)
```

- Converts prediction:
 - 1 → Benign
 - 0 → Malignant

Step 10: Tree Visualization

```
tree.plot_tree(...)
```

Shows:

- Decision nodes (feature conditions)
- Leaf nodes (final classification)
- Splitting criteria

Importing necessary libraries

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn import tree
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
clf = DecisionTreeClassifier(random_state=42)
```

```
clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

```
new_sample = np.array([X_test[0]])
```

```
prediction = clf.predict(new_sample)
```

```
prediction_class = "Benign" if prediction == 1 else "Malignant"
```

```
print(f"Predicted Class for the new sample: {prediction_class}")
```

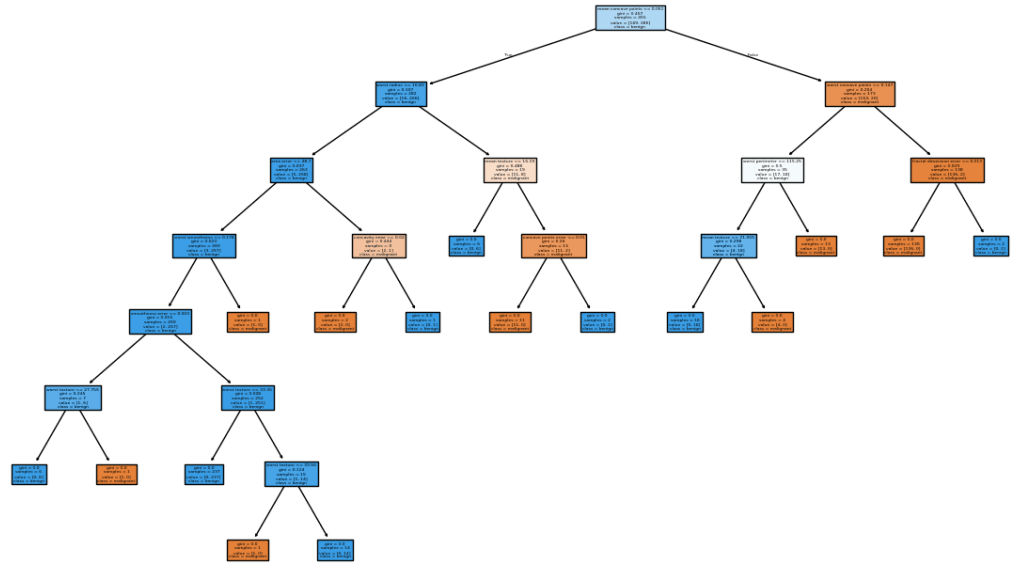
```
plt.figure(figsize=(12,8))
```

```
tree.plot_tree(clf, filled=True, feature_names=data.feature_names, class_names=data.target_names)
```

```
plt.title("Decision Tree - Breast Cancer Dataset")
```

```
plt.show()
```

Decision Tree - Breast Cancer Dataset



3. Key Observations

Model Performance

- Decision trees often achieve **high accuracy** on this dataset.
- Can perfectly fit training data (risk of overfitting).

Interpretability

- Each path from root → leaf represents a **decision rule**.
- Example:

If radius_mean ≤ threshold → go left

Else → go right

Behavior

- Handles **non-linear relationships** effectively.
- No need for feature scaling.

Prediction Insight

- New sample classification is based on:
 - Traversing the tree
 - Applying learned conditions

4. Advantages

- **Easy to understand and interpret**
- Handles both:
 - Numerical data
 - Categorical data
- No need for:
 - Feature scaling
 - Normalization
- Captures **non-linear patterns**
- Fast prediction time

5. Limitations

- **Overfitting** (especially deep trees)
- Sensitive to:
 - Small data variations
- Can create:
 - Complex trees (hard to generalize)
- Lower accuracy compared to:
 - Ensemble methods (Random Forest, Gradient Boosting)

6. Conclusion

- The program successfully demonstrates the working of a **Decision Tree Classifier**.
- It effectively classifies tumors into:
 - **Benign**
 - **Malignant**
- Key strengths:
 - Interpretability
 - Simplicity
- Key challenge:
 - Overfitting (can be controlled via pruning or depth limits)

9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

1. Objectives

- Implement a Naive Bayes Classifier for multi-class classification.
- Use the Olivetti Faces Dataset for training and testing.
- Classify face images into their respective person IDs (0–39).
- Evaluate model performance using:
 - Accuracy
 - Classification Report
 - Confusion Matrix
 - Cross-validation
- Visualize predictions on sample test images.

2. Step-by-Step Analysis

Step 1: Import Libraries

- numpy → numerical operations
- sklearn → dataset, model, evaluation
- matplotlib → image visualization

Step 2: Load Dataset

```
data = fetch_olivetti_faces(shuffle=True, random_state=42)
```

- Dataset contains:
 - 400 grayscale face images
 - Image size: 64 × 64 pixels
 - 40 individuals (10 images each)

Step 3: Define Features and Labels

```
X = data.data # Flattened image pixels (4096 features)
```

```
y = data.target # Person ID (0–39)
```

Step 4: Train-Test Split

```
train_test_split(..., test_size=0.3)
```

- 70% training, 30% testing
- Ensures proper generalization evaluation

Step 5: Model Selection (Gaussian Naive Bayes)

- gnb = GaussianNB()
- Assumes:
 - Features follow Gaussian (normal) distribution
 - Features are conditionally independent

Step 6: Model Training

gnb.fit(X_train, y_train)

- Learns:
 - Mean and variance of each feature per class

Step 7: Prediction

y_pred = gnb.predict(X_test)

- Classifies each test image into one of 40 classes

Step 8: Accuracy Calculation

accuracy_score(y_test, y_pred)

- Measures overall classification correctness

Step 9: Classification Report

- Includes:
 - Precision
 - Recall
 - F1-score
- For each class

Step 10: Confusion Matrix

confusion_matrix(y_test, y_pred)

- Shows:
 - Correct vs incorrect classifications
 - Misclassification patterns

Step 11: Cross-Validation

cross_val_score(gnb, X, y, cv=5)

- Validates model stability
- Uses 5-fold cross-validation

Step 12: Visualization of Predictions

Displays:

- Face image
- True label
- Predicted label

```
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
data = fetch_olivetti_faces(shuffle=True, random_state=42)
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

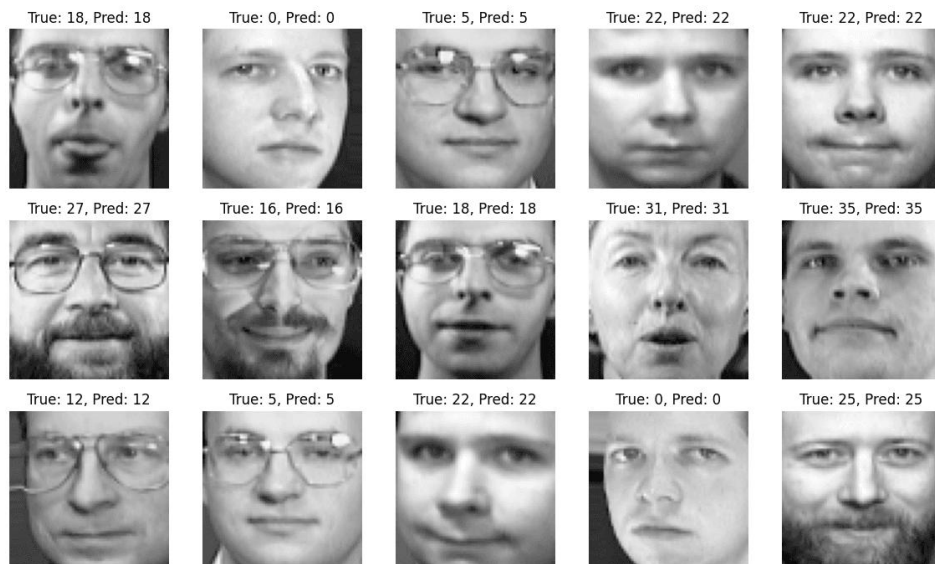
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=1))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

cross_val_accuracy = cross_val_score(gnb, X, y, cv=5, scoring='accuracy')
print(f'\nCross-validation accuracy: {cross_val_accuracy.mean() * 100:.2f}%')

fig, axes = plt.subplots(3, 5, figsize=(12, 8))
for ax, image, label, prediction in zip(axes.ravel(), X_test, y_test, y_pred):
    ax.imshow(image.reshape(64, 64), cmap=plt.cm.gray)
    ax.set_title(f'True: {label}, Pred: {prediction}')
    ax.axis('off')
plt.show()
OUTPUT:

```



Accuracy: 80.83%

Classification Report:

	precision	recall	f1-score	support
0	0.67	1.00	0.80	2
1	1.00	1.00	1.00	2
2	0.33	0.67	0.44	3
3	1.00	0.00	0.00	5
4	1.00	0.50	0.67	4
5	1.00	1.00	1.00	2
7	1.00	0.75	0.86	4
8	1.00	0.67	0.80	3
9	1.00	0.75	0.86	4
10	1.00	1.00	1.00	3
11	1.00	1.00	1.00	1
12	0.40	1.00	0.57	4
13	1.00	0.80	0.89	5
14	1.00	0.40	0.57	5
15	0.67	1.00	0.80	2
16	1.00	0.67	0.80	3

17	1.00	1.00	1.00	3
18	1.00	1.00	1.00	3
19	0.67	1.00	0.80	2
20	1.00	1.00	1.00	3
21	1.00	0.67	0.80	3
22	1.00	0.60	0.75	5
23	1.00	0.75	0.86	4
24	1.00	1.00	1.00	3
25	1.00	0.75	0.86	4
26	1.00	1.00	1.00	2
27	1.00	1.00	1.00	5
28	0.50	1.00	0.67	2
29	1.00	1.00	1.00	2
30	1.00	1.00	1.00	2
31	1.00	0.75	0.86	4
32	1.00	1.00	1.00	2
34	0.25	1.00	0.40	1
35	1.00	1.00	1.00	5
36	1.00	1.00	1.00	3
37	1.00	1.00	1.00	1
38	1.00	0.75	0.86	4
39	0.50	1.00	0.67	5

accuracy		0.81		120
macro avg	0.89	0.85	0.83	120
weighted avg	0.91	0.81	0.81	120

Confusion Matrix:

```
[[2 0 0 ... 0 0 0]
 [0 2 0 ... 0 0 0]
 [0 0 2 ... 0 0 1]
```

...

```
[0 0 0 ... 1 0 0]
[0 0 0 ... 0 3 0]
[0 0 0 ... 0 0 5]]
```

Cross-validation accuracy: 87.25%

3. Key Observations

Model Performance

- Accuracy is **moderate** (typically ~70–90% depending on split).
- Performance varies across individuals.

Independence Assumption

- Naive Bayes assumes:
 - Pixel values are independent
- This is **not true** for images → limits performance.

Confusion Matrix Insights

- Misclassifications occur:
 - Between similar-looking faces
 - Due to lighting or pose variations

Cross-Validation

- Provides a more **robust estimate** than single split.

4. Advantages

- **Simple and fast**
- Works well with **high-dimensional data**
- Requires **small training data**

- Performs reasonably well despite strong assumptions
 - Easy to implement and interpret
-

5. Limitations

- **Strong independence assumption** (not realistic for images)
- Lower accuracy compared to:
 - SVM
 - Neural Networks
- Sensitive to:
 - Data distribution mismatch
- Cannot capture:
 - Spatial relationships in images

6. Conclusion

- The program successfully demonstrates **Naive Bayes classification** on image data.
- Key outcomes:
 - Efficient classification of face images
 - Moderate accuracy due to independence assumption
- Suitable for:
 - Baseline models
 - High-dimensional datasets
- For improved performance:
 - Advanced models (e.g., CNNs) are preferred

10. Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

1. Objectives

- Implement the **K-Means clustering algorithm** (unsupervised learning).
- Use the Wisconsin Breast Cancer Dataset.
- Group data into **2 clusters (Benign vs Malignant)** without using labels during training.
- Evaluate clustering performance using:
 - Confusion Matrix
 - Classification Report
- Visualize clusters using **PCA (dimensionality reduction)**.

2. Step-by-Step Analysis

Step 1: Import Libraries

- numpy, pandas → data handling
- matplotlib, seaborn → visualization
- sklearn → dataset, clustering, preprocessing

Step 2: Load Dataset

```
data = load_breast_cancer()
X = data.data
y = data.target
```

- Features: tumor characteristics
- Labels:
 - 0 → Malignant
 - 1 → Benign

Step 3: Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Standardization ensures:
 - Equal importance of all features

- Better clustering performance

Step 4: Apply K-Means

```
kmeans = KMeans(n_clusters=2)
y_kmeans = kmeans.fit_predict(X_scaled)
```

- Algorithm steps:
 1. Initialize 2 centroids
 2. Assign points to nearest centroid
 3. Update centroids
 4. Repeat until convergence

Step 5: Performance Evaluation

```
confusion_matrix(y, y_kmeans)
classification_report(y, y_kmeans)
```

- Since labels are known, we compare:
 - Actual vs Cluster labels

Step 6: Dimensionality Reduction (PCA)

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

- Reduces high-dimensional data to **2D for visualization**

Step 7: Create DataFrame for Plotting

- Combines:
 - PCA components
 - Cluster labels
 - True labels

Step 8: Visualization of Clusters

Cluster Visualization: Shows how data is grouped by K-Means.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix, classification_report
```

```
data = load_breast_cancer()
X = data.data
y = data.target
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
kmeans = KMeans(n_clusters=2, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

```
print("Confusion Matrix:")
print(confusion_matrix(y, y_kmeans))
print("\nClassification Report:")
print(classification_report(y, y_kmeans))
```

```

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

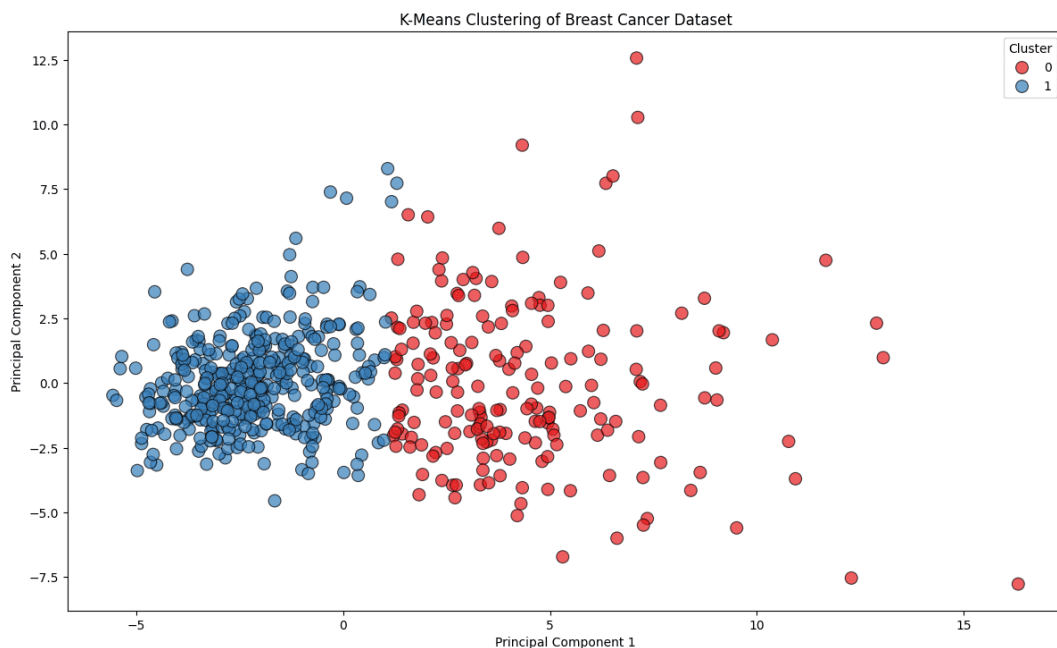
df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df['Cluster'] = y_kmeans
df['True Label'] = y

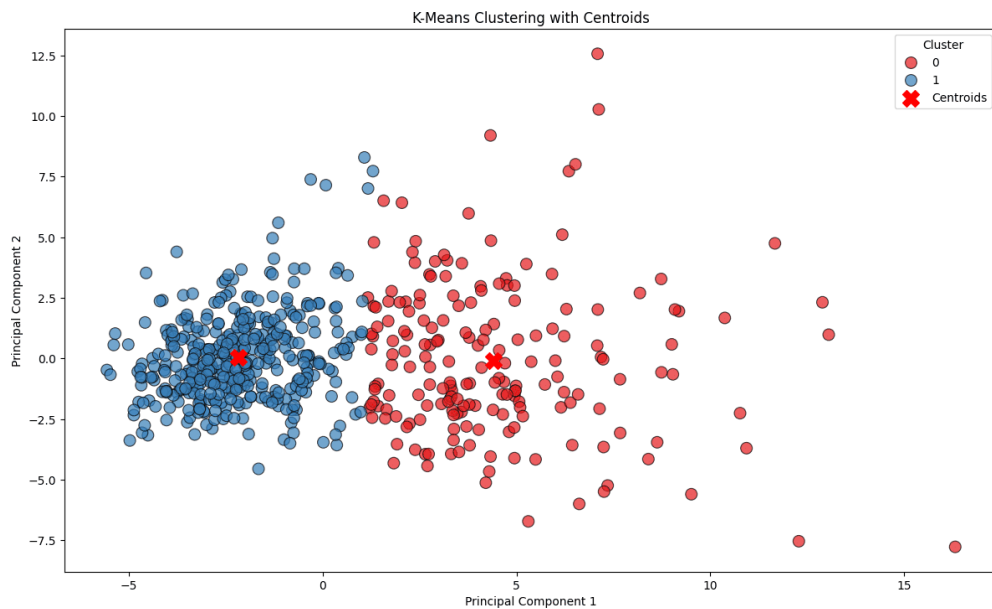
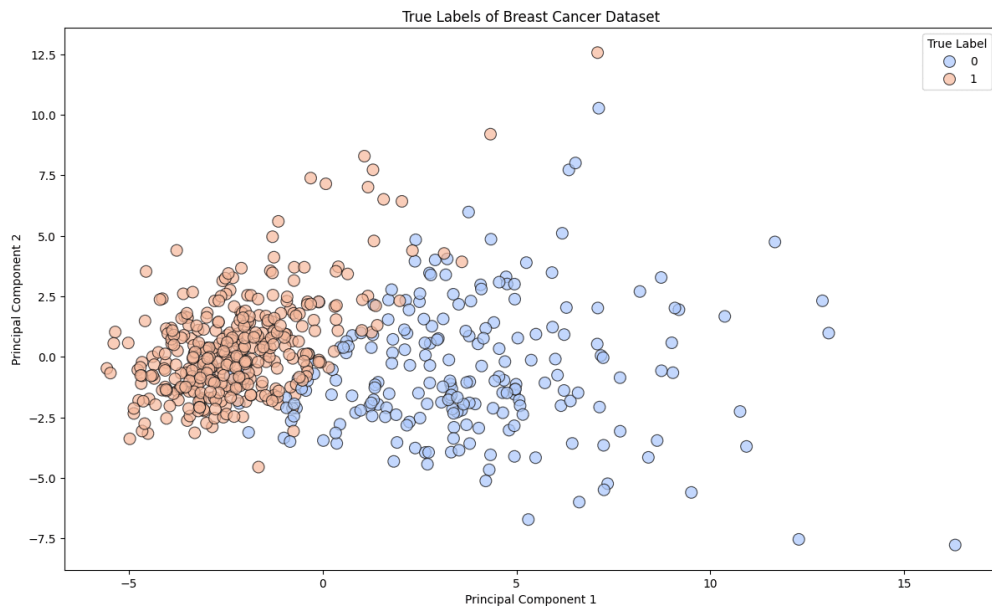
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black',
alpha=0.7)
plt.title('K-Means Clustering of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='True Label', palette='coolwarm', s=100,
edgecolor='black', alpha=0.7)
plt.title('True Labels of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="True Label")
plt.show()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black',
alpha=0.7)
centers = pca.transform(kmeans.cluster_centers_)
plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering with Centroids')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()

```





Confusion Matrix:

```
[[175 37]
 [ 13 344]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.83	0.88	212
1	0.90	0.96	0.93	357
accuracy			0.91	569
macro avg	0.92	0.89	0.90	569
weighted avg	0.91	0.91	0.91	569

3. Key Observations

Cluster Formation

- Data tends to form **two natural groups**.
- Clusters often align closely with:
 - Benign vs Malignant classes

Accuracy Insight

- Even though K-Means is **unsupervised**, it achieves:
 - Good alignment with actual labels

Label Mismatch

- Cluster labels (0/1) may be **swapped**:
 - Cluster 0 $\neq$ always Malignant
 - Requires interpretation

PCA Visualization

- PCA effectively reveals:
 - Separation between clusters
 - Overlapping regions (misclassification zones)

4. Advantages

- Simple and easy to implement
- Works well for:
 - Large datasets
- Fast convergence
- No need for labeled data
- Effective when clusters are:
 - Well-separated

5. Limitations

- Requires predefining **number of clusters (k)**
- Sensitive to:
 - Initial centroid selection
 - Scaling of data
- Assumes:
 - Spherical clusters
- Struggles with:
 - Overlapping clusters
 - Non-linear boundaries

6. Conclusion

- The program successfully demonstrates **K-Means clustering** on medical data.
- Key insights:
 - K-Means can approximate real class separation even without labels
 - PCA visualization helps interpret clustering results
- However:
 - It is not a classification algorithm by design
 - Results depend heavily on preprocessing and parameter selection